01 / WHAT WE FOUND

Memglide: what improved

The fixes passed validation. Source-only Memglide showed a promising cost estimate, but a combined cost-and-speed advantage is still unproven.

1,443
distinct automated cases verified

60 / 60
strict successes in the fresh confirmation

120
native attempts retained across both studies

What changed in the product

More accurate freshness checks. Only real source and memory dependencies invalidate saved context. Unrelated filename matches no longer trigger false invalidation.

Safer session handling. Retrieval, saved context, events and activity updates commit together. Ended or expired sessions cannot receive late retrieval updates.

Less unnecessary local reading. Irrelevant memories are filtered before source validation. A local fixture reduced secure reader calls **55 → 15**, with the same selected evidence. This does not measure model-token or dollar savings.

Bounded context and honest accounting. Optional support files use complete, validated reads and a shared size cap. Unsafe or incomplete preparation falls back to the original prompt. Unverified cumulative usage is rejected by cost accounting.

What passed

1,340 Python + 24 frontend + 3 Chromium cases passed in the full application validation; **76 new benchmark cases** passed separately. Installed CLI, API, web app and both MCP modes also passed functional checks. Lint and type checks passed.

The enlarged Python suite was collected, not rerun in full. Earlier failures and their fixes remain in the evidence. Concurrent test runs now use isolated temporary directories.

The first 60 benchmark attempts remain strict failures. Their behavior checks passed, but agents changed protected existing tests. Instructions were clarified and ten new tasks were tested; the old results were never regraded. Application source was unchanged between studies.

02 / MEASURED COMPARISON

Cost estimate down; time up

Fresh confirmation: 10 tasks (5 Python, 5 TypeScript), 2 repeats and 3 configurations = 60 runs. Same gpt-6.1-sol model and settings, matched starting commits and task instructions, isolated worktrees. Full time includes cold Memglide approval and indexing.

A = standard Codex · **B** = Memglide source only
C = source plus caller-selected supporting tests

All confirmation tasks, including controls

Setup	Strict successes	Estimated $ per success	Mean seconds	Median seconds
A	20 / 20	$0.02996	20.970	19.515
B	20 / 20	$0.02922	21.196	20.431
C	20 / 20	$0.03054	22.545	21.121

Focused tasks: where source retrieval can help

Eight tasks repeated twice; all 16 runs per configuration passed. Four provided-context control runs per configuration are excluded only from this focused view.

Setup	Estimated $ per success	Mean seconds	Median seconds
A	$0.03122	20.222	19.515
B	$0.02804	20.865	21.300
C	$0.03046	22.775	21.121

B vs A, focused: 10.19% lower estimated cost per success, but 3.18% longer mean time. Across all confirmation tasks, B's estimated cost advantage is only 2.47%.
C vs B, focused: 8.66% higher estimated cost and 9.16% longer mean time.

Observed mean token usage: all confirmation tasks

Setup	Total input	Cached input[1]	Uncached input	Output
A	39,559	30,272	9,287	836
B	38,414	29,299	9,114	806
C	41,006	31,584	9,422	854

[1] Cached input is part of total input. Dollar estimates use observed usage at standard API-equivalent rates per million tokens: $2 input, $0.10 cached reads, $2.50 cache writes and $10 output. Cache-write attribution remains uncertain. Actual charges and subscription savings were unavailable.

03 / DECISIONS & LIMITS

What to keep, what to test next

Keep source-only as the default. Keep support files optional. Support failed 17 of 24 predeclared cost/time checks. On focused tasks it reduced recognized read commands (1.125 → 0.25 per run), but total commands rose (3.69 → 4.63), as did uncached input, output and measured local CPU.

Why the signals are mixed

A removed read can be replaced by other work. Cached input is 20 times cheaper than ordinary input at the listed rates, so fewer total tokens need not mean a lower cost. Local CPU was a lower-bound measurement; sampled process memory did not rise proportionally and excludes some indexing overhead.

Every failed attempt remains in cost per success

Campaign totals include both studies: 40 attempts per configuration, with 20 strict successes and 20 failures each. Different task sets are not pooled to claim a performance gain.

Setup	Estimated spend, all 40 attempts	Estimated $ per strict success
A	$1.28895	$0.06445
B	$1.28260	$0.06413
C	$1.24747	$0.06237

What is still unproven

Actual money savings: dollars are list-price equivalents. Unknown cache-write attribution puts B's focused saving scenarios between **−3.54% and +22.49%**; this is not a confidence interval.

MCP and retrieval scope: live comparisons used source preload, with **zero configured MCP calls**. MCP functionality passed separately. Selected-source freshness and completeness passed, but this does not establish whole-repository retrieval recall.

Generality: small purpose-built repositories and two repeats per task. Provider and OS caches were uncontrolled; byte-identical controls also varied. Model-call counts, actual file-read counts and server resources were unavailable. Large-repository and cross-session gains remain untested.

Next engineering priorities

1. Obtain supported per-request billing, cache-write and model-call telemetry.
2. Test fresh related tasks across sessions and larger unfamiliar repositories, with independent grading.
3. Retain cheap conditional bypasses; promote extra retrieval only when it improves cost per success and runtime without reducing correctness.

Detailed evidence

Full results & individual-run links · Validation · Research & architecture references
Download complete evidence ZIP — includes failed runs and reproducible JSON cost reports.